Explain the project

Source checked: 21 October 2026. This guide covers Cyber Grid, not Pocket Console. Line numbers refer to the current files and can move after later edits.

Your opening explanation

“Cyber Grid is a browser-based tic-tac-toe game using HTML, CSS, and JavaScript. It supports local multiplayer, player versus AI, and AI versus AI. Players can choose Classic or Vanishing Marks, and the first to three round wins takes the match. Visual effects provide feedback, while local saving lets players continue an unfinished match.”

Where to open the code

All file references in this PDF are relative to this exact project folder:

```
C:\Users\Admin\Documents\Codex\2026-09-18\can-we-add-something-more-that\outputs\grid-duel
```

Example: **script.js:638** means open script.js inside that folder, press **Ctrl+G** in VS Code, type **638**, and press Enter. Use **Ctrl+F** with the function name if lines have changed.

Files to study first

1. **index.html:** board, buttons, title screen, settings.
2. **script.js:** rules, AI, controller, saving, and effects.
3. **style.css:** layout, colors, responsiveness, animations.
4. **server/server.js and server/db.js:** optional backend and database.

Explain the architecture in one sentence

“The browser handles gameplay and presentation; localStorage keeps the unfinished match on that browser; the Express backend stores completed match records in SQLite.”

Important distinction

Opening index.html directly runs the frontend. To use completed-match database saving, run **npm start** from the project folder and open **http://localhost:3001**. Node.js and installed dependencies are required. The leaderboard endpoint exists, but there is no leaderboard interface in the current frontend.

```
package.json:3 | starts node server/server.js
```

Our browser stack

HTML - structure and interaction

Defines the board, buttons, and dialogs. We chose native HTML elements for a simple, accessible page structure.

`index.html:70 | nine-square board container`

`index.html:115 | loads the game controller`

CSS - appearance and layout

Controls the cyberpunk theme, sizing, and animations. Separating styling from rules makes changes easier to manage.

`style.css:6 | shared theme variables`

`style.css:125 | impact movement animation`

`style.css:1240 | responsive layout for smaller screens`

Vanilla JavaScript - game behavior

Controls rules, turns, AI, scores, and events without a frontend framework. This project is small enough to use browser APIs directly and demonstrate programming fundamentals.

`script.js:638 | main move controller`

`script.js:80 | validates and creates the next rules state`

SVG - sharp lines and shapes

Draws scalable bolts, circuitry, and the winning line. Vector shapes remain sharp when resized. SVG is a browser graphics technology, not a separate game engine.

`index.html:26 | lightning SVG`

`index.html:81 | winning-line SVG`

Canvas 2D - continuously drawn effects

Draws the circuit background and some effects. Canvas gives us a drawing surface that JavaScript can redraw without creating an HTML element for every particle.

`index.html:20 | background canvas element`

`script.js:1854 | circuit renderer gets its canvas`

Web Audio API - synthesized sound

Generates tones directly, avoiding separate audio files. Sound remains optional.

`script.js:760 | creates the audio context`

`script.js:765 | creates a tone and gain control`

Storage, backend and tools

localStorage - unfinished match

A browser key-value store that survives closing the page. We use JSON to store moves, round results, and settings. It is simple and needs no server, but it is browser-specific and can be cleared.

`script.js:2114 | save key: cyber-grid-unfinished-v1`

`script.js:2145 | writes or removes the unfinished save`

`script.js:2115 | validates and replays saved moves`

`script.js:2161 | restores the match in a paused state`

Node.js - backend JavaScript runtime

Runs server code outside the browser. Using JavaScript on both sides avoids introducing another programming language for a small backend.

`package.json:3 | npm start command`

Express - HTTP routes

Handles browser requests and serves the game files. We chose it to keep the small API straightforward.

`server/server.js:1 | imports Express`

`server/server.js:15 | accepts a completed match`

`server/server.js:35 | returns leaderboard data`

`script.js:43 | frontend sends the completed-match request`

SQLite + better-sqlite3 - completed results

SQLite stores structured records in one local database file. better-sqlite3 connects Node.js to it. This is practical for a small project without a separate database service.

`server/db.js:1 | database driver`

`server/db.js:8 | opens data/grid-duel.db`

`server/db.js:12 | schema: player, mode, winner, scores, timestamp`

`server/server.js:27 | parameterized insert; values passed separately`

Development tools and supporting assets

VS Code is the editor. **Git** tracks source changes. **GitHub** hosts the repository. These are development tools, not code running the game. The package file records the repository URL. Google Fonts and Font Awesome provide typefaces and icons, not game logic.

`package.json:9 | repository metadata`

`index.html:9 | external fonts`

`index.html:13 | external icon stylesheet`

Game logic you must know

The move flow

Click → validate → copy/update state → check result → update score/display → save → next turn. The board is an array of nine strings. An empty string is a free square; X or O is an occupied square.

`script.js:638 | connects the rules to the interface`

Wins and draws

There are eight winning combinations: three rows, three columns, and two diagonals. A round win gives one point; three points win the match. Classic draws when the board is full. Vanishing Marks also detects three repeated positions and a 60-move limit. A victory is checked before draw conditions.

`script.js:67 | eight winning combinations`

`script.js:80 | win/draw checks`

Vanishing Marks: a queue remembers age

Each player keeps up to three marks. The destination must first be legal and empty. On a fourth placement, remove that player's oldest mark, add the new mark, then check victory. This order prevents a disappearing mark from counting in a winning line. Repetition checks include the next turn and both queues, not only the visible board.

`script.js:85 | removes the oldest mark`

AI: explain minimax simply

“The AI tries future moves and compares their outcomes, assuming the opponent also plays well.” Easy chooses randomly about 70% of the time; Medium about 20%. Hard always searches. Classic searches to the end; Vanishing Hard searches eight moves ahead, while the other search levels use four. Vanishing uses an evaluation score and alpha-beta pruning to avoid exploring unnecessary branches. Do not claim Vanishing Hard is unbeatable.

`script.js:112 | difficulty and search selection`

Why copying state matters

The rules return a new state instead of changing the old one. This lets the AI test hypothetical moves without changing the actual game. The ScoreBoard class holds the scores across rounds.

`script.js:58 | constructor, addWin, and reset`

Why old CPU moves do not appear after a reset

A timeout delays the CPU move. Cancelling it and checking a revision number prevents a callback from an older round from making a move. Returning to the title pauses play; reopening restores the save and waits for Continue.

`script.js:716 | timer and revision guard`

`script.js:2213 | pause and title navigation`

Practice your defense

Why no React?

The scope is small enough for vanilla JavaScript. It also makes DOM events, arrays, functions, and state changes easier to demonstrate.

Why SQLite instead of MySQL?

We need a small local database. SQLite has a simple file-based setup. A larger multi-user deployment might require a different architecture.

Is Continue stored in SQLite?

No. Unfinished matches are in localStorage. SQLite stores completed matches through the backend.

What is your main limitation?

Multiplayer is local, saves do not sync across devices, and the server does not independently verify an entire game. It is not an anti-cheat ranking system.

How did you verify it?

Automated checks covered win/draw rules, illegal moves, AI branches, Vanishing rounds, timer cleanup, and save/load behavior. These do not replace manual browser and visual testing.

Did you use AI assistance?

Yes. AI tools helped develop and refine the project. I must understand the implementation, verify behavior, and explain the choices honestly.

Programming requirements: quick code tour

`script.js:58 | Class and methods`

`script.js:67 | Array of winning patterns`

`script.js:80 | Conditionals and state objects`

`script.js:112 | Loops and reusable functions`

`script.js:103 | Higher-order functions`

`script.js:674 | Console output`

Five-minute rehearsal

Explain the stack. Make one legal move and trace it in code. Demonstrate the oldest mark disappearing. Explain minimax without claiming the AI learns. Pause, close, reopen, and continue using the same browser. Demonstrate backend saving only when the server is running.

Pocket Console is a separate edition: it currently has no Express/SQLite backend. Its browser storage saves preferences, not the unfinished Cyber Grid match. Do not mix the two implementations when answering.